Prepared Statements are enabled by default on PostgreSQL. You can disable prepared statements by setting `prepared_statements` to `false`:

```
production:
  adapter: postgresql
  prepared_statements: false
```

If enabled, Active Record will create up to `1000` prepared statements per database connection by default.

To modify this behavior you can set `statement_limit` to a different value:

```
production:
  adapter: postgresql
  statement_limit: 200
```

The more prepared statements in use: the more memory your database will require. If your PostgreSQL database is hitting memory limits, try lowering `statement_limit` or disabling prepared statements.

3.13.4 Configuring an SQLite3 Database for JRuby Platform

If you choose to use SQLite3 and are using JRuby, your `config/database.yml` will look a little different. Here's the development section:

```
development:
  adapter: jdbcsqlite3
  database: db/development.sqlite3
```

3.13.5 Configuring a MySQL Database for JRuby Platform

If you choose to use MySQL and are using JRuby, your `config/database.yml` will look a little different. Here's the development section:

```
development:
  adapter: jdbcmysql
  database: blog_development
  username: root
  password:
```

3.13.6 Configuring a PostgreSQL Database for JRuby Platform

If you choose to use PostgreSQL and are using JRuby, your `config/database.yml` will look a little different. Here's the development section:

```
development:
  adapter: jdbcpostgresql
  encoding: unicode
  database: blog_development
  username: blog
  password:
```

Change the username and password in the `development` section as appropriate.

3.14 Creating Rails Environments

By default Rails ships with three environments: "development", "test", and "production". While these are sufficient for most use cases, there are circumstances when you want more environments.

Imagine you have a server which mirrors the production environment but is only used for testing. Such a server is commonly called a "staging server". To define an environment called "staging" for this server, just create a file called `config/environments/staging.rb`. Please use the contents of any existing file in `config/environments` as a starting point and make the necessary changes from there.
That environment is no different than the default ones, start a server with `rails server -e staging`, a console with `rails console staging`, `Rails.env.staging?` works, etc.

3.15 Deploy to a subdirectory (relative url root)